

Grayson Reflection

Hannah and I worked together on the threads portion of the project. Our process was highly collaborative throughout, and having dedicated in-class time to work together was really valuable, as it gave us time to think through the problem together before splitting off to write code individually. Our general workflow started with high-level discussion: we talked through the requirements of the scenario, aligned on how the synchronization logic should work, and decided how to divide up the files/work in a way that would be easy to integrate. Agreeing on the shared state structure and the signals between cats and mice upfront meant we could work independently without running into issues later on.

When it came to actually coding, we'd each write our respective files, then come back together to review what we'd written, discuss any inconsistencies or changes, and combine everything onto one laptop before committing to GitHub. This approach worked well for keeping the code consistent and avoiding the overhead of constantly pushing, merging, and pulling when working on interconnected files. Since Hannah ran into some issues on her machine over the course of the semester we worked primarily on my laptop, though we divided the actual writing evenly.

My primary contributions to the threads implementation were `simulation.h` and `cat.c`. In `simulation.h`, I defined the shared state structure (configuration parameters, semaphores for bowl occupancy, the active cat counter, and the mutex and condition variables that coordinate cat and mouse behavior). Getting this file right was important early on since it served as the contract between our two files; once Hannah and I agreed on it, we could write `cat.c` and `mouse.c` independently, and Hannah could also work on `main.c`. In `cat.c`, I implemented the cat thread lifecycle: the not-hungry rest period, waiting for a free bowl, updating the active cat count on arrival and departure, broadcasting to mice, and feeding for the configured duration.

I also worked on defining the testing structure, which was a new experience for me. I'm quite comfortable writing unit tests in Java and C++ to verify the behavior of individual functions, but didn't have much experience designing large-scale scenario tests. Brainstorming and running configurations with varying numbers of bowls, cats, mice, and durations, and then interpreting the output, gave me a better intuition for how concurrency bugs manifest in practice, not as crashes, but as subtle violations of liveness or safety that only show up under specific timing conditions.

Beyond my pair with Hannah, working with Anushka and Vivian as a full group was honestly so much fun. Even though much of the implementation happened in smaller pairs, the group stayed communicative and aligned throughout. We checked in regularly, shared progress, gave each other feedback, and there was a collaborative spirit to the project that made it feel like a shared effort rather than parallel isolated work. Overall, I found this project to be super rewarding and fun, both for what I learned about synchronization and concurrency, and for how well the team dynamic worked and how much I enjoyed the group project structure!